

MITIT 2025 Winter Contest Editorials: Beginner Round

The MITIT Organizers

January 19, 2025

1 MIT Time

Problem Idea: Sam Zhang

Problem Preparation: Andrii Smutchak

Analysis By: Albert Wang

If $1 \leq N \leq 5$, then we are on MIT time. Otherwise, we can iterate k from $2, \dots, 13$ and check if N lies in the interval $(5^{k-1}, 5^k]$, and if it does, we are on MIT^k time.

Note that it is impossible for $k > 13$, as $5^{13} \geq 10^9 \geq N$.

2 M(IT)+

Problem Idea: Albert Wang

Problem Preparation: Brendon Jiang

Analysis By: Albert Wang

We will focus on each M in the input. Each M must be followed by some positive number of copies of the string IT in order for the string to be valid.

Therefore, we may loop over the string. Every time we see an M , we scan the substring following it up to the next M or the end of the string, and verify that it is some positive number of repetitions of IT . This check takes $O(N)$ time, giving a linear time algorithm.

3 Traveling Salesman Problem

Problem Idea: Arul Kolla

Problem Preparation: Arul Kolla

Analysis By: Albert Wang

For each point (x, y) let $f(x, y) = x + y$. We can see that the original statement is equivalent to traveling salesman problem in one dimension where point i is now located at $f(x_i, y_i)$ on the number line. It is now clear that it is optimal to start from the minimum $f(x_i, y_i)$ and visit points in ascending order of $f(x_i, y_i)$, returning to the minimum $f(x_i, y_i)$ at the end. Afterwards, we can just cyclically shift so that we start and end at city 1. Each interval between adjacent points in the one dimensional case will be traversed twice, so our final answer is just

$$2 \cdot (\max_i (x_i + y_i) - \min_i (x_i + y_i)).$$

4 Scoreboard Screenshots

Problem Idea: Ditbul Ban

Problem Preparation: Ditbul Ban

Analysis By: Ditbul Ban

4.1 Subtask 1

Note that there is only one team, so if we sort the screenshots by the nondecreasing order of the score, the ordering will satisfy the condition, and it would be a valid answer. This solution only involves sorting, so the subtask could be solved in $O(N \log N)$.

4.2 Subtask 2

For every pair of screenshots (p, q) , we could check if it is possible for screenshot q to appear after screenshot p . If there exists any team i such that $A_{p,i} > A_{q,i}$, then scoreboard q cannot appear after scoreboard p , so we can determine that scoreboard q should be placed before scoreboard p .

Iterating this for all scoreboard pairs will give us a directed graph between scoreboards. Then, the problem becomes equivalent to providing an ordering of the vertices such that if an edge $u \rightarrow v$ exists, then v comes after u . This is exactly the same as the topological sort problem, so we can use the algorithm to solve this problem in $O(N^2K)$. Here, the time complexity comes from the comparison of every pair of screenshots.

Actually, we can do better than this. If scoreboard q appears before scoreboard p , it means that for all i , $A_{p,i} \leq A_{q,i}$. This also means that the sequence A_q should be lexicographically larger (or same) than A_p . Therefore, if we sort all indices by the lexicographical order of A_i , either we get a valid ordering of the screenshots, or there is no valid ordering. Sorting will take $O(NK \log N)$ time, and we can check the validity of the solution in $O(NK)$; therefore, we have a $O(NK \log N)$ solution. Observing that it is enough to sort the indices by the value of $\sum_{1 \leq j \leq K} A_{i,j}$ will yield a $O(NK)$ time solution.

Since it is difficult to distinguish between a fast $O(N^2K)$ solution and a slow $\tilde{O}(NK)$ solution, the problem was prepared so that either solution could get a full score.

5 Missing Number Queries

Problem Idea: Sam Zhang

Problem Preparation: Sam Zhang

Analysis By: Sam Zhang

For the first subtask, we can directly answer the queries by sorting all the elements in the query interval and looking for missing numbers.

For the second subtask, if the given array has some number x between 1 and N missing from it, we may answer all queries using x . Otherwise, the array must be a permutation of the numbers 1 through N . In that case, we can answer each query using any element of the array outside the query interval.

To solve the full problem, we need a way to keep track of whether the array is a permutation, after possibly changing some of its elements. We may keep track of the frequency of each value 1 through N in the array; when performing a type 1 operation, we decrement the frequency of the old value and increment that of the new value at the position. At the same time, we can keep a list of all values with frequency zero, say, in a `std::set` in C++. When answering a type 2 query, we check if any frequency zero elements exist. If so, output any such element; otherwise output any element in the array not in the query interval.

6 AAB-BAA

Problem Idea: Yifan Kang

Problem Preparation: Yifan Kang

Analysis By: Albert Wang

Note that the move is equivalent to swapping any two letters that are a distance 2 apart. This means that we can split the string into its even and odd positions, and transform each separately into the target. If the counts of letters in the even positions does not match the counts of letters in the even positions of the target string, it is impossible, and similar for odd positions.

Now, let's consider how many swaps are required to solve the even positions — our problem is now effectively figuring out how many adjacent swaps are needed to transform a binary string to a target binary string. In particular, focus on the B s. There is no reason to change the order of the B s, and if a particular B moves a distance of d , it must do so in d swaps. Noting that a swap can only move one B at a time, we can compute for each k the distance between the k -th B in the starting string and the k -th B in the ending string, and the sum of these values must be the number of swaps that are needed. This sum can be computed in $O(N)$ time.

For the odd positions, we may use analogous logic. Adding up the answers for both even and odd positions gives us the desired answer, and our overall complexity is still $O(N)$.

7 Grid and Numbers Game

Problem Idea: Sam Zhang

Problem Preparation: Dylan Isaac

Analysis By: Dylan Isaac

Notice that after any arbitrary sequence of moves, the relative ordering of all entries of the grid remains the same, i.e. if two adjacent cells c, c' have $c < c'$ at the beginning, then they must have $c < c'$ for the rest of the game.

Consider a state from which no further moves can be made. Such a grid must have the same relative ordering as the starting grid, and every cell has a neighbor that is exactly one less than it. In particular, this grid is unique, as these conditions imply that every cell will be equal to the length of the longest path from it to a local minimum. We can construct it as follows:

- Iterate over cells in increasing order of value. Then, let each cell's new value be 1 more than the maximum new value of any cell surrounding it, or 0 if it has no neighbors that are less than it (i.e. a local minimum.)

Noting that the game must terminate, we conclude that all games, regardless of moves, must terminate at this ending grid. Furthermore, we know exactly how many moves it takes to do so; the sum of the grid decreases by 1 on each move.

Therefore, we can simply find the length of any sequence of moves that reach this end-state, and check its parity to determine the winner. If the sequence of moves has an odd length, Busy Beaver moved last and therefore wins. Otherwise, Calico Bear wins.

We can now construct the ending grid as described above and compute the answer in $O(NM \log(NM))$, with the log factor due to sorting.

8 Toy Marbles

Problem Idea: Ditbul Ban

Problem Preparation: Ditbul Ban

Analysis By: Ditbul Ban

This problem is shared with Advanced Round 2 Problem 1.

8.1 Subtask 1

Let's call the first type of action as **swap** and second type of action as **merge**.

Notice that if all c_i are distinct, then c becomes a permutation of length N . Here, we should not use the merge operation, since each container should have exactly one marble in the end.

Therefore, the problem is equivalent to sorting the permutation c using only swap operations. This is a well-known problem which can be solved using *cycle decomposition*.

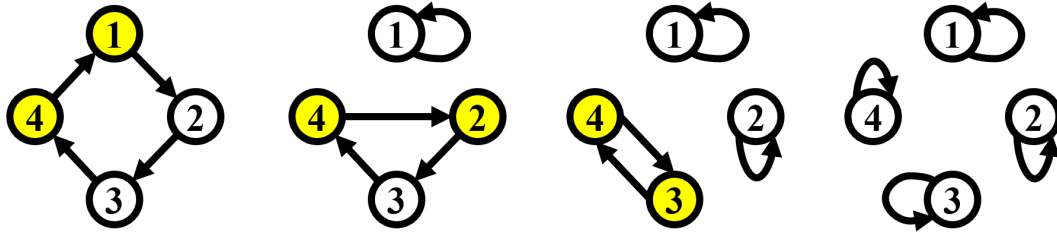
We consider the graph G , which consists N vertices and N edges. For every i s.t. $1 \leq i \leq N$, there is an edge $i \rightarrow c_i$ in the graph. (Notice that each vertex has exactly one outgoing edge; we call these graphs *functional graphs*.) Here, we can think each vertex v has an outgoing edge *pointing* where the marble in container v should actually be included. Notice that each swap operation will swap the destination of two outgoing edges in graph G ; in other words, if there were two edges $x \rightarrow e_x$ and $y \rightarrow e_y$ and we swap two containers x and y , then the two edges are removed and two new edges $x \rightarrow e_y$ and $y \rightarrow e_x$ are newly formed. Our goal is to make every edge into self loops (an edge which the starting and ending vertex is the same), since it will mean that all marbles are in their right place.

Due to the constraints of the subtask, the functional graph should be comprised only by cycles. Let's denote the initial number of cycles as s . In the final state, we should have N cycles. We could easily observe the following fact:

- Swapping two vertices in the same cycle increases the number of cycles by 1.
- Swapping two vertices *not* in the same cycle decreases the number of cycles by 1.

Obviously, we only want the number of cycles to increase, since we should make it to the maximum. Therefore, each time we should swap two vertices in the same cycle, splitting one cycle into two. Therefore we could conclude that the minimum number of operations we need is $N - s$.

After all of these observations, constructing a solution is not difficult. One simple way is iterating for every cycles, and separate all vertices in each cycle to self loops. Below is an example image of how to separate all vertices in a cycle to self loops. This can be done in $O(N)$.



8.2 Subtask 2

In this subtask, c_i are not necessarily distinct. The solution is quite intuitive, but proving the correctness is very complicated. First let's start with the following observation.

Observation 8.1. *In the optimal solution, the number of merge operations is fixed.*

Proof. Observe that each merge operation **usually** decreases the number of nonempty containers by 1. Therefore, if all merge operations are usual in this sense, then the optimal solution will contain $n - s$ merge operations, where s is the number of nonempty containers in the final state.

The unusual cases is when either x or y is empty in the merge operation. If x is empty, then this is equivalent to a swap operation, so we will consider this as a swap operation. If y is empty, then nothing happens, and we don't need this operation. Therefore, we could conclude that at least one optimal solution would not contain any unusual merge operations, and thus have exactly $n - s$ merge operations. $\square$

Therefore, the crucial part of solving this problem is to minimize the number of swap operations. We can observe one more property, which will simplify our analysis.

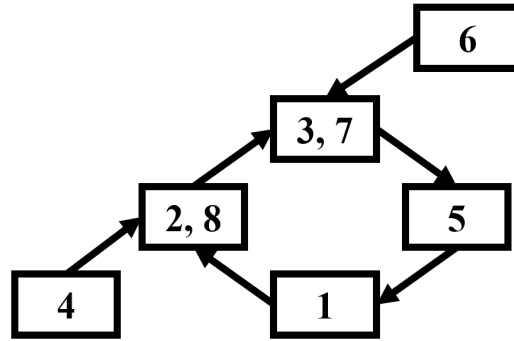
Observation 8.2. *There is an optimal solution where all the merge operations occur before all of the swap operations.*

Proof. We omit the rigorous proof here, but we will rather provide an intuition for this fact. Imagine that a merge operation happened after a swap operation. If the swap operation is independent with the merge operation (that is, they do not affect the same containers), then we can just move the merge operation to the front. If the swap operation affects a container which is also affected by the merge operation, we could accordingly change the index of the merge operation, and move it to the front. $\square$

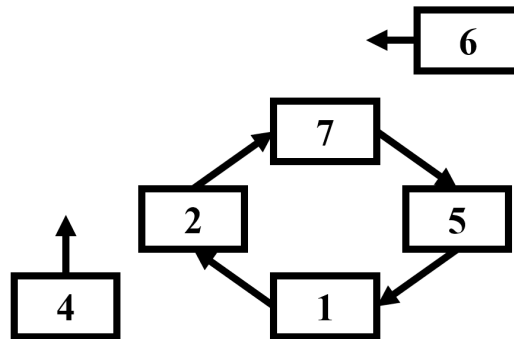
If we consider that the merge operation happens before the swap operations, then this problem becomes very similar to Subtask 1. However, we have to decide in which container to merge the marbles of each color. To solve this problem, let's think about the following situation.

- First, partition $\{1, 2, \dots, N\}$ to one or more sets in a way such that u and v is in the same set if and only if $c_u = c_v$. We will call this partition P .
- Second, we will create a directed graph G' such that each vertex corresponds to a set in P . Each vertex will have exactly one outgoing edge, which points towards the set in P which contains the c_i value.

For example, if $N = 8$ and $c = [2, 7, 5, 8, 1, 3, 5, 7]$, the graph G will be constructed as the image below.



Notice that this graph is *similar* to the graph G in Subtask 1. The difference is that in Subtask 1, each vertex represented only one marble, but here, each vertex could represent multiple marbles. Also, recall the fact that if multiple marbles have same color, then they could be merged in any of the containers. In the above example, $c_2 = c_8 = 7$, so the two marbles can be merged in either container 2 or 8. If we decide where to merge all the marbles, then we can directly construct graph G in Subtask 1. For example, if we move marble 8 to container 2, and move marble 3 to marble 7, we can get graph G as follows (here, marble x means the marble initially in container x):



Here, notice that vertex 4 or 6 has outgoing edges which are not pointing any vertex. This is because there are no longer any marbles in the containers they have to reach. For example, marble 4 has to move to container 8, but there are no marbles in container 8.

If we finish our decision in the merging process and get this graph G , let's figure out how many swaps we need to accomplish our goal. The graph G could be decomposed into cycles and chains which ends at a node pointing nowhere. We already analyzed the cycles in Subtask 1; we need $k - 1$ swap operations, if the cycle contains k vertices. For the chains, there are no cycles at first, but each swap operation inside the chain will increase the number of cycles by 1. This will yield a method of changing a chain of length l to l cycles in l swap operations. In conclusion, if there are $|V|$ vertices on graph G' (which should be same as the number of different colors in c), and there are k cycles in the graph, we will need $|V| - k$ swap operations to finish the process.

(For a rigorous proof, we should also analyze the cases where we perform a swap operation between cycles and chains, two different chains, or two different cycles. Analyzing each case will show that they do not do better than the methods we showed above.)

To minimize $|V| - k$, we should know about the maximum possible value of k ; in other words, how many cycles could G contain?

Claim 8.3. *The maximum number of cycles of G is equal to the number of cycles of G' .*

Proof. If we imagine the process of converting graph G' into G , it should be obvious that we cannot have more cycles in G , so this fact gives an upper bound for k .

We can also show that if we decide well on the merging process, we can construct graph G which has the same number of cycles with G' . The intuition is that we should **preserve** the cycles in G' . If a cycle in G' contains a node representing A (a set of containers), and the node has an outgoing edge which points the node representing B (another set of containers), B will contain the destination x of all marbles in A (which is equal to the colors of all marbles in A). If we merge all marbles in B to container x , the cycle will be preserved. $\square$

Combining all these facts would give a method to use the minimum number of merge and swap operations, thus giving an optimal solution. This can be done in $O(N)$.

9 Snakes on a Grid

Problem Idea: Sam Zhang

Problem Preparation: Duru Ozer

Analysis By: Duru Ozer

This problem is shared with Advanced Round 2 Problem 2.

9.1 Subtask 1

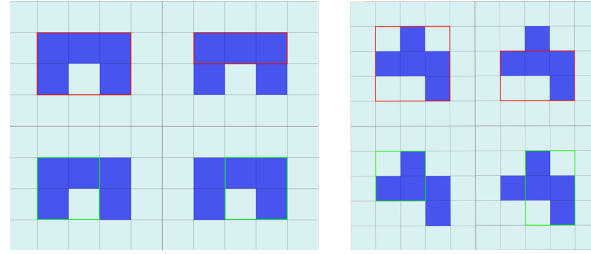
For this subtask it is enough to write a brute force solution checking whether each connected component in the queried rectangle is a snake. There are various ways to do this. One such way is to find connected components by using a DSU (Disjoint Set Union) structure. Then to check for being snakes, we could find a way to sort the cells in each component so they are in the order they would appear in a snake shape. This can be achieved by sorting the cells in the component by their coordinate pairs. Note that for a valid snake, there are two possible sortings: either both coordinates increasing / decreasing together or one coordinate increasing and the other one decreasing. We can try both of them and see if one of them works. To check if a 0-indexed sorted array of size n is a snake, it is sufficient to check for 3 things:

1. $[(r_i - r_{i-1}), (c_i - c_{i-1})] = [(r_{i-2} - r_{i-3}), (c_{i-2} - c_{i-3})]$ for $3 \leq i \leq n - 1$ i.e. The difference between adjacent cells repeats every 2 steps
2. $|r_i - r_{i-1}| + |c_i - c_{i-1}| = 1$, i.e, the distance between each adjacent cell pair is 1
3. $[(r_1 - r_0), (c_1 - c_0)] = [(r_2 - r_1), (c_2 - c_1)]$, i.e. The component is not a line (only if $n > 2$)

So we can try sorting the cells in both ways and checking for these 3 conditions. If all 3 of them hold for at least one sorting, the component is a snake. Total time complexity : $O(A \log A)$, where A is the sum of areas over all queries.

9.2 Subtask 2 & 3

The key observation behind subtasks 2 and 3 is that for every non-snake component, there are $O(1)$ rectangles partially or fully contain that component and include a non-snake piece of it. In the following figure, such rectangles are marked by red.



We can find all such rectangles for each component in the grid in $O(1)$, thus total complexity over the grid being $O(NM)$. Once we find them, a queried rectangle is good if and only if it doesn't contain any of the non-snake rectangles. We can check this by using 2D prefix sums. Specifically, we can use a different prefix sum for different rectangle sizes. For example, for Subtask 2, these sizes would be $[(1, 4), (2, 2), (2, 3), (3, 2), (2, 2), (4, 1)]$. Note that the number possible sizes for the defined rectangles for both subtasks is constant. Thus the total complexity is $O(NM + Q)$ with the constant differing between subtasks.

9.3 Full Solution

The key observation to the full solution is that, if a component is not a snake, it either contains 3 cells in a row or 4 cells in a 2×2 square. This can easily be checked in $O(1)$ by using 3 2D prefix sums:

1. One for checking 3 horizontally adjacent cells with the same value
2. One for checking 3 vertically adjacent cells with the same value
3. One for checking 2×2 squares with the same value

Namely, for each of the 3 unwanted shapes, we can choose a designated cell to mark, for instance the middle cell for the 3-in-a-row cases and the top left corner for the 2×2 square case. Then we can mark those cells wherever such a shape exists, and adjust the boundaries for the prefix sum so that if a cell is marked and is inside the boundaries, the unwanted shape is present in the queried rectangle.